

Experiment # 04

Understanding and implementing KNN algorithm for classification problem

Objectives:

- Overview of KNN
- To understand working principals of KNN
- How to select a good value of K
- How KNN can be used for regression and classification both
- Impact of imbalanced and outliers on KNN
- Importance on scaling dataset
- Advantages and disadvantages of KNN
- Example
- Lab Tasks

1. Overview of KNN

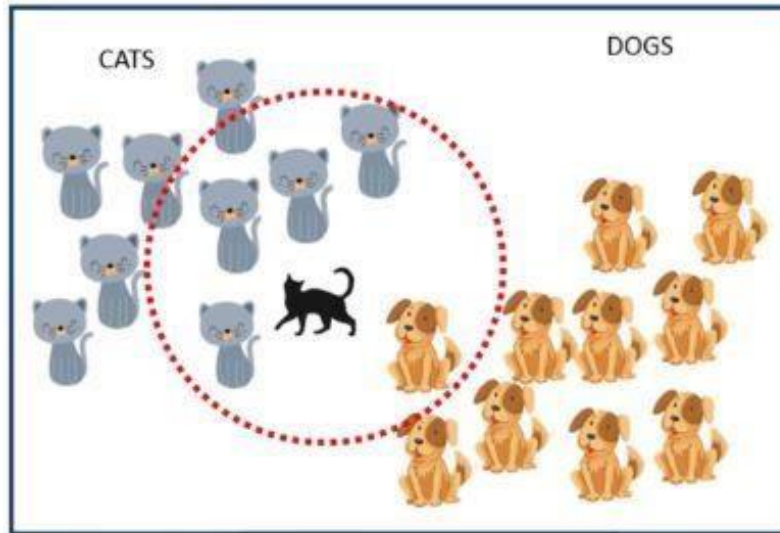
K-Nearest Neighbour is one of the simplest machine learning algorithms based on supervised learning technique. It assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for *Regression* as well as for *Classification* but mostly it is used for the classification problems.

K-NN is a *non-parametric algorithm*, which means it does not make any assumption on underlying data. It is also called a *lazy learner algorithm* because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Consider the diagram below; it is straightforward and easy for humans to identify it as a “Cat” based on its closest allies. This operation, however, cannot be performed directly by the algorithm. KNN calculates the distance from all points in the proximity of the unknown data and filters out the ones with the shortest distances to it. As a result, it’s often referred to as a *distance-based algorithm*.

In order to correctly classify the results, we must first determine the value of K (Number of Nearest Neighbours). In the following diagram, the value of K is 5. Since there are four cats and just one

dog in the proximity of the five closest neighbours, the algorithm would predict that it is a cat based on the proximity of the five closest neighbors in the red circle's boundaries.



2. Working principle/algorithm

The K-NN working can be explained on the basis of the below algorithm:

Step-1: Select the number K of the neighbors

Step-2: Calculate the Euclidean distance of all the neighbours from the test point

$$\text{distance} = \sqrt{\sum_{i=1}^n (p1[i] - p2[i])^2}$$

Step-3: Sort all the neighbours based on their distances

Step-4: Take the K nearest neighbors as per the calculated Euclidean distance.

Step-5: Among these k neighbors, count the number of the data points in each category.

Step-6: Assign the new data points to that category for which the number of the neighbor is maximum.

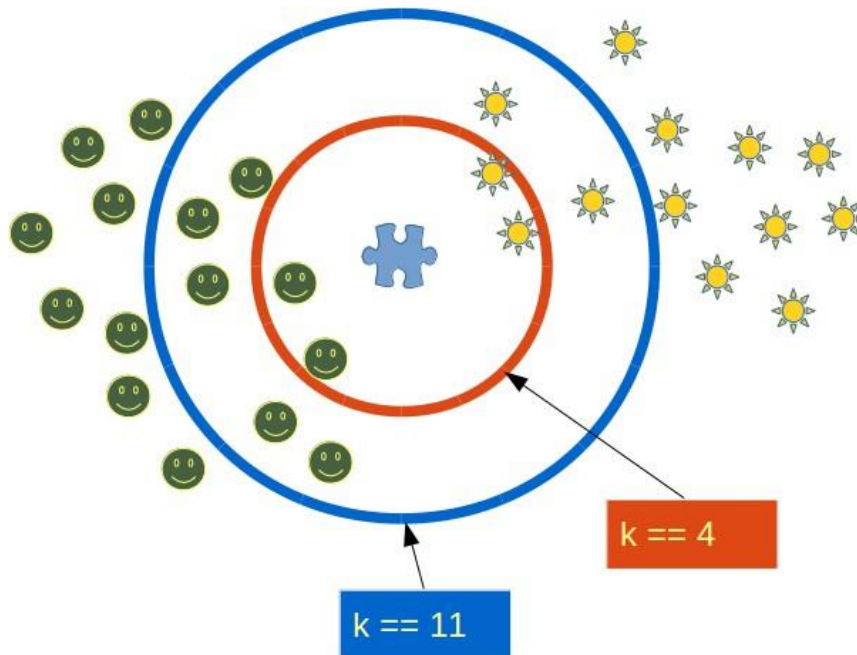
3. Fine tuning value of K

We don't have a particular method for determining the correct value of K. Although, we can try to test the model's accuracy for different K values. The value of K that delivers the best accuracy for both training and testing data is selected. That can only be possible for small datasets.

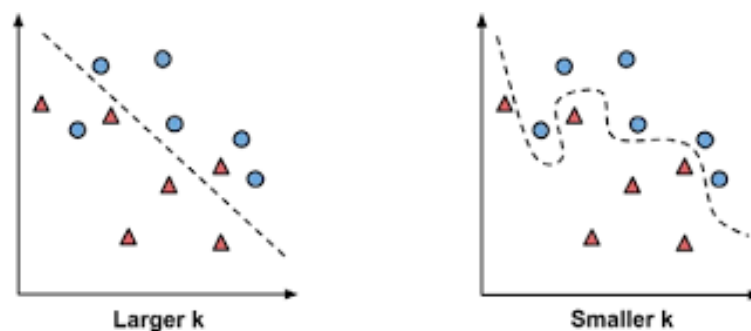
It is recommended to always select an odd value of K. When the value of K is set to even, a situation may arise in which the elements from both groups are equal. In the diagram below, elements from both groups are equal in the internal "Red" circle (K == 4).

In this condition, the model would be unable to do the correct classification. Here the model will randomly assign any of the two classes to this new unknown data.

Choosing an odd value for K is preferred because such a state of equality between the two classes would never occur here. Due to the fact that one of the two groups would still be in the majority, the value of K is selected as odd.



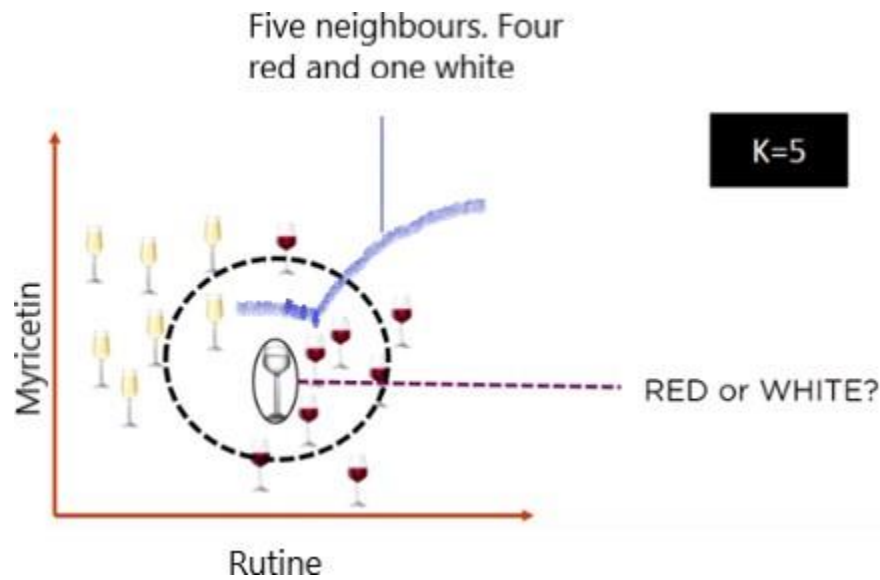
The impact of selecting a smaller or larger K has their own effects on the algorithm. Larger K value leads to *underfitting*. In this case, the model would be unable to correctly learn on the training data. Likewise, smaller K value may cause *overfitting* where the model will capture all of the training data, including noise. The model will perform poorly for the test data in this scenario. Refer to the picture below



4. KNN used for classification and regression

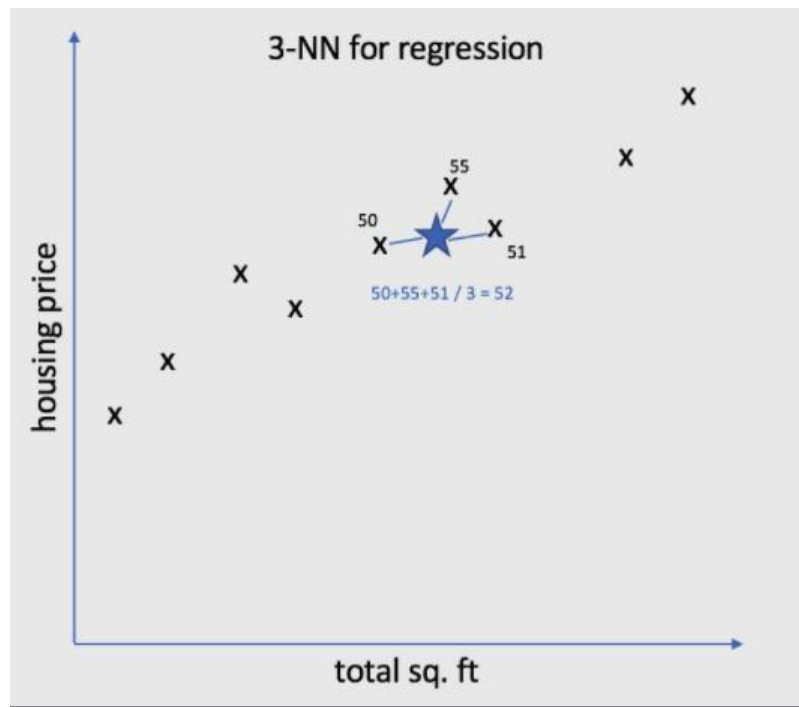
When the problem statement is of ‘**classification**’ type, KNN tends to use the concept of “*Majority Voting*”. Within the given range of K values, the class with the most votes is chosen.

Consider the following diagram, in which a circle is drawn within the radius of the five closest neighbours. Four of the five neighbours in this neighbourhood voted for ‘RED,’ while one voted for ‘WHITE.’ It will be classified as a ‘RED’ wine based on the majority votes.



In case of **regression**, KNN employs a mean/average method for predicting the value of new data. Based on the value of K , it would consider all of the nearest neighbours. The algorithm attempts to calculate the mean for all the nearest neighbours' values until it has identified all the nearest neighbours within a certain range of the K value.

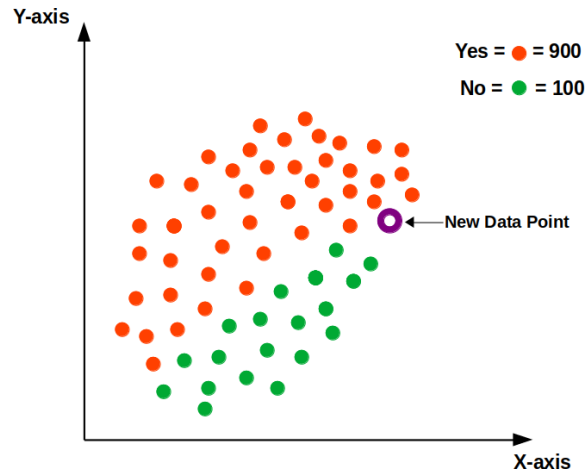
Consider the diagram below, where the value of k is set to 3. It will now calculate the mean (52) based on the values of these neighbours (50, 55, and 51) and allocate this value to the unknown data.



5. Impact of imbalanced and outliers on KNN

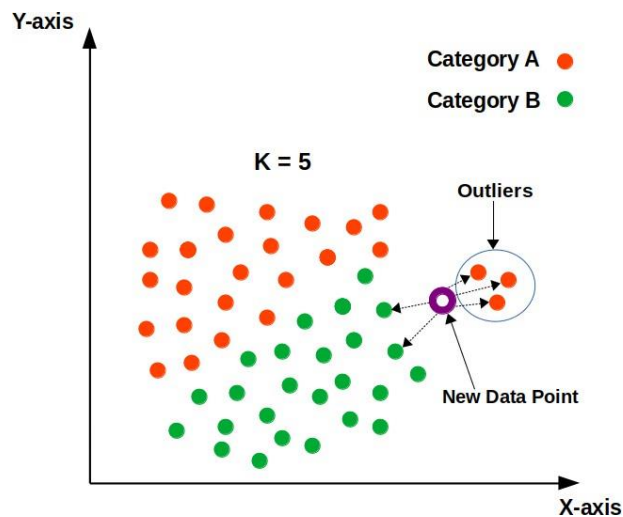
5.1. Imbalanced data

When dealing with an imbalanced data set, the model will become biased. Consider the example shown in the diagram below, where the “Yes” class is more prominent. As a consequence, the bulk of the closest neighbours to this new point will be from the dominant class. Because of this, we must balance our data set using different strategy.



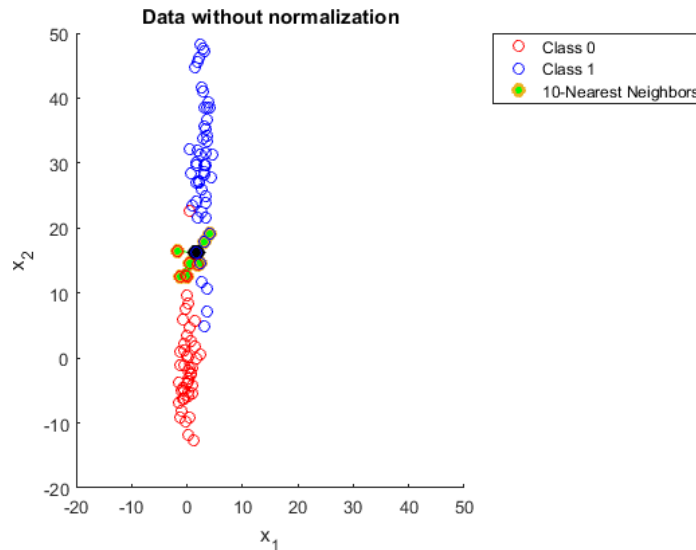
5.2. Outliers

Outliers are the points that differ significantly from the rest of the data points. The outliers will impact the classification/prediction of the model. The appropriate class for the new data point, according to the following diagram, should be “Category B” in green. The model, however, would be unable to have the appropriate classification due to the existence of outliers. As a result, removing outliers before using KNN is recommended.

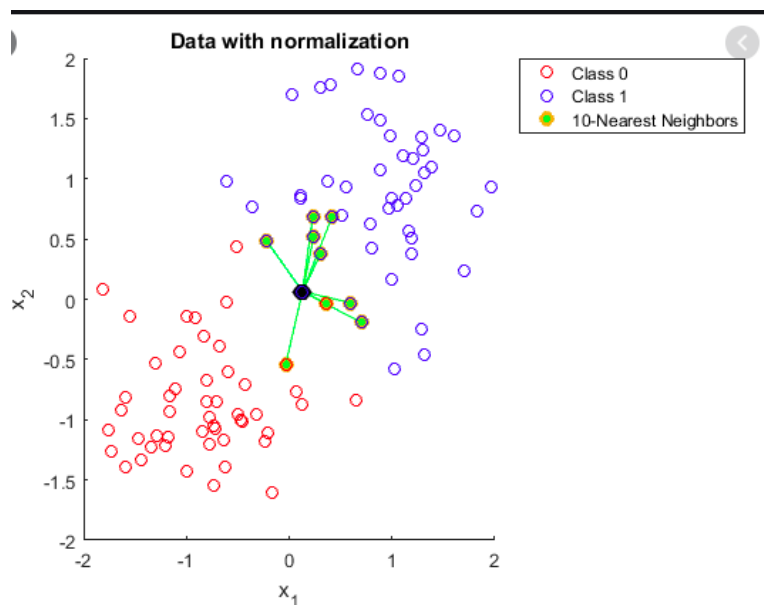


6. Importance of scaling dataset

Data has always two attributes: Magnitude and Unit. For instance; if we say 20 years then “20” is the magnitude here and “years” is its unit. Since it is a distance-dependent algorithm, KNN selects the neighbours in the closest vicinity based solely on the magnitude of the data. Have a look at the diagram below; the data is not scaled, so it cannot find the closest neighbours correctly. As a consequence, the outcome will be influenced.



The data values in the previous figure have now been scaled down to the same level in the following example. Based on the scaled distance, all of the closest neighbours would be accurately identified.



7. Pros and cons of KNN

7.1. Pros:

1. Since the KNN algorithm requires no training before making predictions, new data can be added seamlessly, which will not impact the accuracy of the algorithm.
2. KNN is very easy to implement. There are only two parameters required to implement KNN—the value of K and the distance function (e.g. Euclidean, Manhattan, etc.)

7.2. Cons:

1. The KNN algorithm does not work well with large datasets. The cost of calculating the distance between the new point and each existing point is huge, which degrades performance.
2. Feature scaling (standardization and normalization) is required before applying the KNN algorithm to any dataset. Otherwise, KNN may generate wrong predictions.

8. Example

Following example is a classification problem solved by using KNN and implemented for IRIS dataset.

Importing libraries

```
import numpy as np
from scipy.stats import mode
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from numpy.random import randint
```

Calculating the Euclidean Distance

```
# Euclidean Distance
def euclidian(p1, p2):
    dist = np.sqrt(np.sum((p1 - p2) ** 2))
    return dist
```

Function to find KNN and then predict

```
def predict(x_train, y, x_input, k):
    op_labels = []

    # Loop through the Datapoints to be classified
    for item in x_input:

        # Array to store distances
        point_dist = []

        # Loop through each training Data
        for j in range(len(x_train)):
            distances = euclidian(np.array(x_train[j, :]), item)
            # Calculating the distance
            point_dist.append(distances)
        point_dist = np.array(point_dist)

        # Sorting the array while preserving the index
        # Keeping the first K datapoints
```

Department of Computer Engineering
CS-302 Artificial Intelligence

```
dist = np.argsort(point_dist)[:k]

# Labels of the K datapoints from above
labels = y[dist]

# Majority voting
lab = mode(labels)
lab = lab.mode[0]
op_labels.append(lab)

return op_labels
```

Here we load IRIS dataset, split test and train and call functions

```
# Loading the Data
iris = load_iris()

# Store features matrix in X
X = iris.data
# Store target vector in
y = iris.target

# Creating the training Data
train_idx = xxx = randint(0, 150, 100)
X_train = X[train_idx]
y_train = y[train_idx]

# Creating the testing Data
test_idx = xxx = randint(0, 150, 50) # taking 50 random samples
X_test = X[test_idx]
y_test = y[test_idx]

# Applying our function
y_pred = predict(X_train, y_train, X_test, 7)

# Checking the accuracy
print(accuracy_score(y_test, y_pred))
```


9. Tasks

1. Write the above-mentioned program for different datasets. Show data head with features used and corresponding labels in case of classification problem. Evaluate results for all metrics.
2. Use KNN for regression dataset. Show data head with features used and corresponding labels in case of classification problem. Evaluate results for all metrics.